

Algoritmos y Estructuras de datos OOP y Punteros en c++



FCEFYN

Punteros



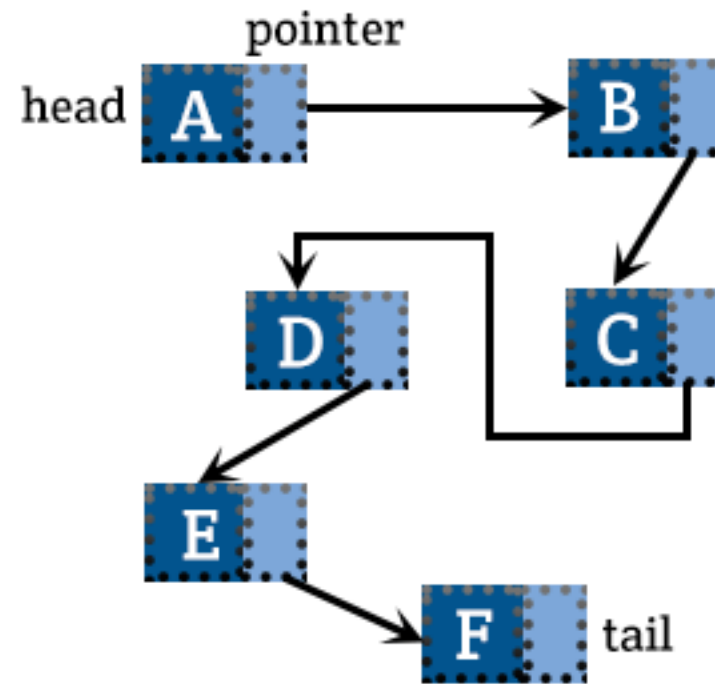
- Un puntero es una variable que almacena la dirección donde reside otro objeto.
- Por ejemplo, para almacenar una lista de elementos, podríamos usar un arreglo contiguo, pero la inserción en medio del arreglo contiguo requiere la reubicación de muchos elementos.
- Es común almacenar cada elemento en una pieza de memoria separada y no contigua, que se asigna mientras el programa se ejecuta.
- Junto con cada objeto hay un enlace al siguiente objeto. Este enlace es una variable de puntero, porque almacena la ubicación de memoria de otro objeto.

Punteros

Array

index	
0	A
1	B
2	C
3	D
4	E
5	F

Linked List



Punteros



```
#include <iostream>

int main() {
    int *ptr = nullptr;
    int x = 10;
    ptr = &x;

    std::cout << "Valor al que apunta el puntero: " << *ptr << std::endl;

    return 0;
}
```

Nullptr se puede utilizar desde c++ 11 , en versiones anteriores se usaba NULL

Punteros



```
double x = 5.5;  
double* px = &x;  
cout << *px << '\n';  
*px = 10.0;  
cout << x << '\n';
```

Declara x = 5.5

Crea un puntero a x en px

Imprime lo apuntado por px -> x --- 5.5

Modifica lo apuntado por px -> x = 10.0

Imprime el valor de x --- 10.0

& → “Operador de dirección”

* → “Operador de indirección” o “operador de desreferencia”

Memoria dinámica

- Generalmente no inicializamos punteros utilizando el operador de dirección (&).
- Los punteros se utilizan para referirse a objetos creados dinámicamente.
- Para crear objetos dinámicamente se utiliza el operador new, y para liberar la memoria asignada a esos objetos, se utiliza el operador delete.

```
double* px = new double;  
*px = 5.1234;
```

Memoria dinámica

```
new double;  
new double(1.23456);  
new std::string(line, 5, 10)
```

Apunta a →



Apunta a → 1.23456

```
double* px = new double;  
*px = 5.1234;  
delete px;
```

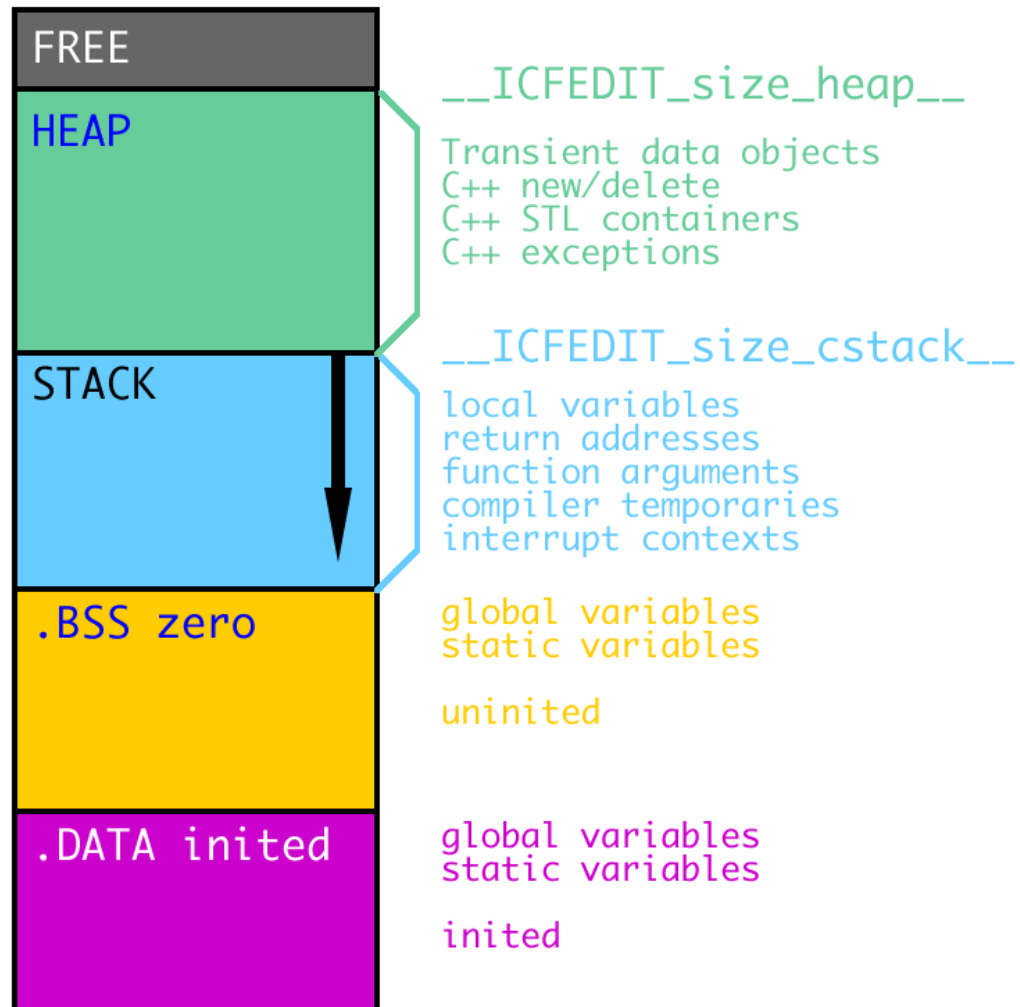
Ejercicio



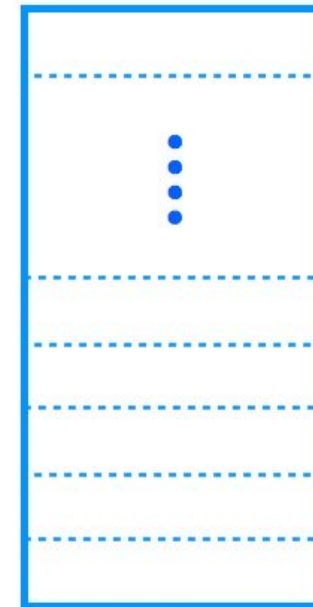
¿Cuál es el valor de cada variable a lo largo de la ejecución ?

```
double x = 5.5;  
double y = 10.2;  
double z;  
double *px = &x;  
double& ry = y;  
*px = 3.14;  
  
z = ry;  
double &r = *px;  
r = 99.44;  
px = &z;  
ry = *px;
```

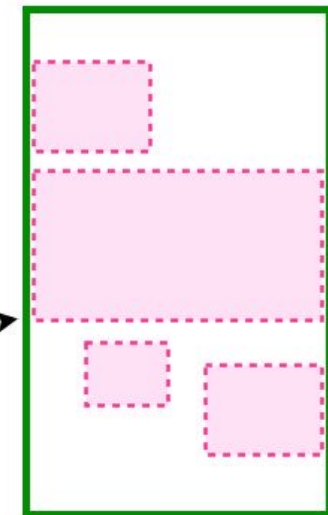
Heap vs Stack



stack



heap



static

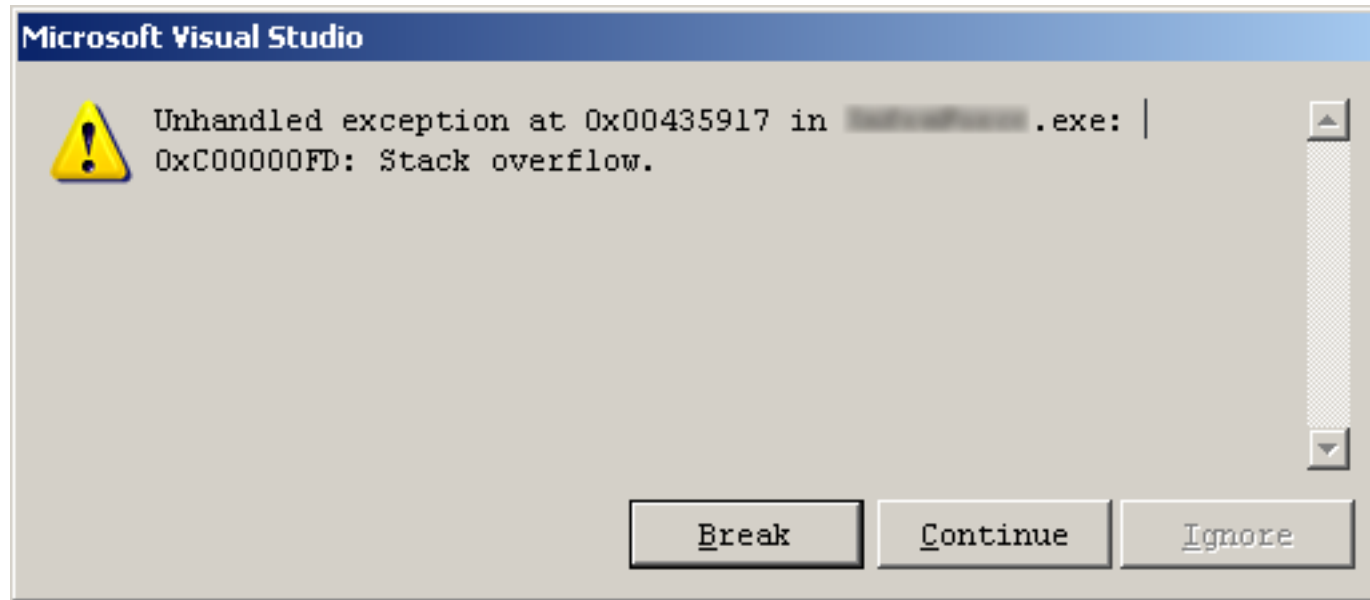


Stack



- La memoria en la pila se organiza en un formato de LIFO (Last In, First Out).
- La pila se utiliza para almacenar variables con un tiempo de vida predecible dentro del programa.
- La asignación y liberación de memoria en la pila son rápidas y automáticas, ya que el sistema operativo maneja estas operaciones.
- El espacio en la pila es limitado y está preasignado, y el tamaño máximo puede variar según la implementación y la configuración del sistema.

Stack overflow



Recursividad



```
int factorial(int n)
{
    return n * factorial(n-1);
}
```



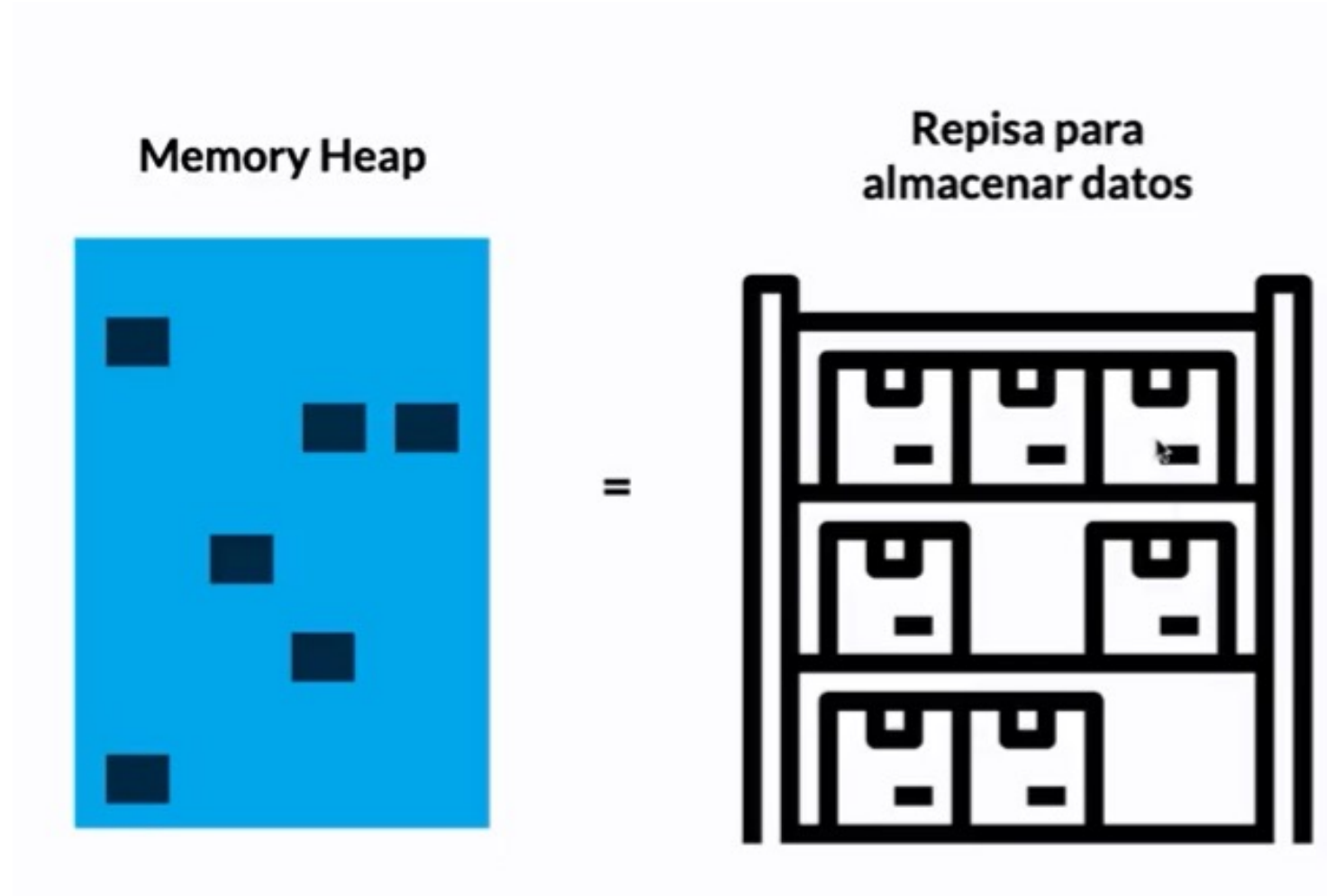
Stack Overflow

Heap



- La memoria en el heap es un área grande y más flexible de la memoria
- Se utiliza para almacenar datos de duración dinámica, cuya duración y tamaño no se conocen en tiempo de compilación, como objetos creados con new.
- La asignación y liberación de memoria en el heap son más flexibles pero también más lentas y requieren más recursos que en la pila.
- La gestión de la memoria en el heap es responsabilidad del **programador**, quien debe asegurarse de liberar la memoria asignada dinámicamente cuando ya no se necesite (usando delete).

Heap



Memory Leaks

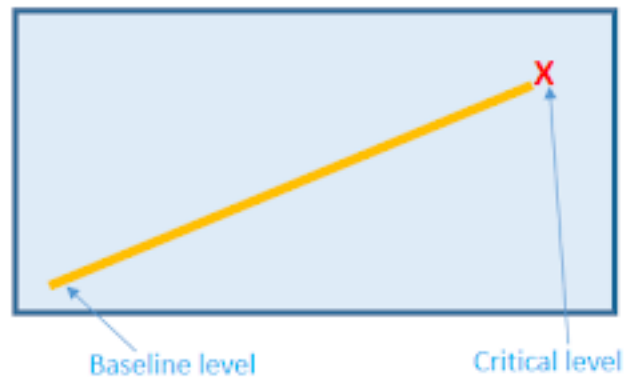


Una fuga de memoria ocurre cuando un programa asigna memoria dinámicamente y no la libera correctamente cuando ya no es necesaria.

Conduce a un desperdicio de memoria, ya que la memoria asignada sigue estando reservada incluso cuando no se utiliza, lo que puede llevar eventualmente a una degradación del rendimiento o incluso a la falta de memoria.

Memory Leaks

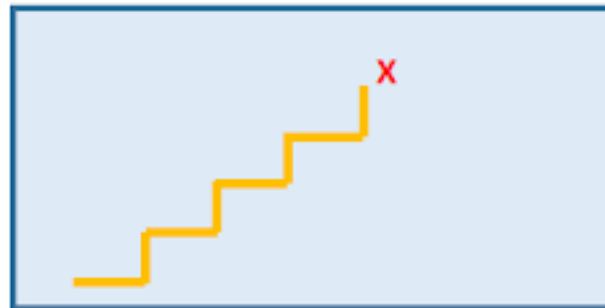
Memory Leak



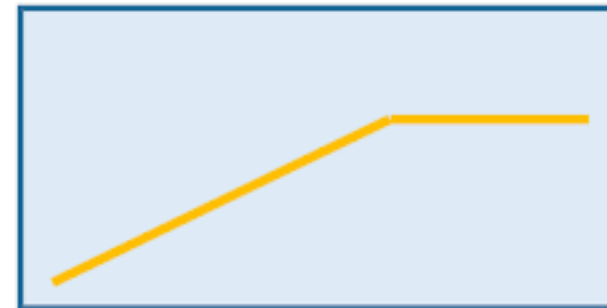
No Leak



Memory Leak



No Leak



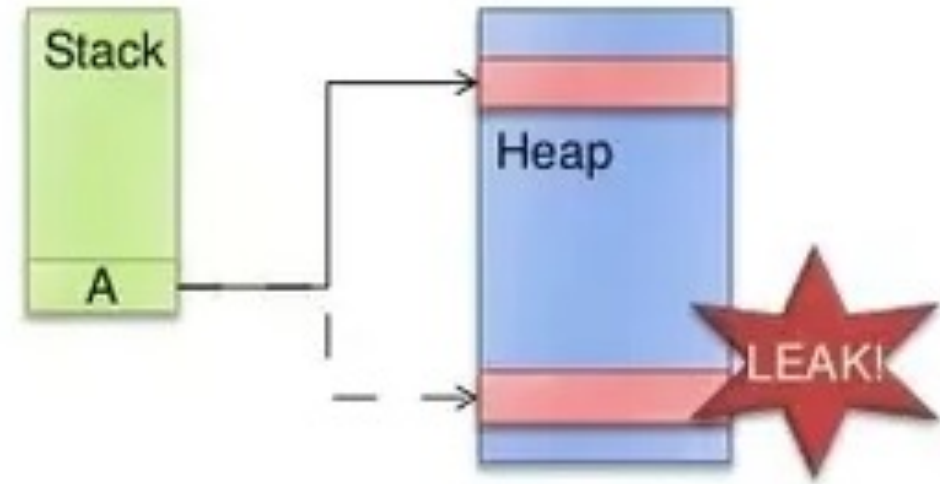
Memory Leaks

```
#include <stdio>

void Foo()
{
    int* a = new int[100];
    a = new int[1000];
}

void Foo1()
{
    int* a = new int[100];
    int* b = new int[1000];
    a = b;
}
```

Leak!





Ejercicio



¿ Qué problema tiene este programa ?

```
1 #include <iostream>
2
3 int main() {
4     for (int i = 0; i < 10; ++i) {
5         int* ptr = new int;
6         *ptr = i;
7         std::cout << "Puntero creado. Valor: " << *ptr << std::endl;
8     }
9
10    return 0;
11 }
```


Structs en c++

- Un struct es una estructura de datos que permite agrupar diferentes tipos de datos bajo un solo nombre.
- Los miembros (variables) dentro de un struct pueden tener diferentes tipos de datos.

```
struct Persona {  
    int edad;  
    string nombre;  
}
```

Structs en c++ - Constructores

- Podemos tener diferentes constructores

```
struct Persona {  
    int edad;  
    string nombre;  
  
    Persona() {  
        edad = 0;  
        nombre = "Sin nombre";  
    }  
  
    Persona(int edadParametro, const string& nombreParametro) {  
        edad = edadParametro;  
        nombre = nombreParametro;  
    }  
};
```

Listas de inicialización

- Se utiliza la sintaxis de ":" y a continuación los campos a inicializar y entre paréntesis sus valores.

```
struct Persona {  
    string nombre;  
    int edad;  
  
    Persona() : nombre("Sin nombre"), edad(0) {  
    }  
  
    Persona(const string& nombreParametro, int edadParametro)  
        : nombre(nombreParametro), edad(edadParametro) {  
    }  
};
```

Structs en c++ - Constructores

- Existen constructores especiales que si no los generamos el compilador los generará por nosotros
 - Constructor de copia
 - Constructor de movimiento

```
Persona(const Persona& otra) : edad(otra.edad), nombre(otra.nombre) {  
    cout << "Constructor de copia llamado" << endl;  
}  
  
Persona(Persona&& otra) noexcept : edad(otra.edad), nombre(move(otra.nombre)) {  
    cout << "Constructor de movimiento llamado" << endl;  
}
```

Structs en c++ - Constructores

- También podemos incluir un destructor y al igual que para el constructor por defecto el compilador incluirá uno por defecto si no lo definimos.

```
struct Persona {  
    char* nombre;  
  
    Persona() : nombre(nullptr) {}  
  
    Persona(const char* nombreParametro) {  
        nombre = new char[strlen(nombreParametro) + 1];  
        strcpy(nombre, nombreParametro);  
    }  
  
    ~Persona() {  
        delete[] nombre;  
    }  
};
```

Clases vs Structs

- Visibilidad Predeterminada:
 - En una clase, los miembros son privados por defecto
 - En un struct, los miembros son públicos por defecto
- Herencia:
 - En una clase, la herencia es privada por defecto, lo que significa que los miembros públicos de la clase base no son accesibles directamente a través de la clase derivada.
 - En un struct, la herencia es pública por defecto, lo que significa que los miembros públicos de la estructura base son accesibles directamente a través de la estructura derivada.

```
class Persona {
public:
    Persona() : edad(0), nombre("Sin nombre") {
        cout << "Constructor por defecto llamado" << endl;
    }

    Persona(int edadParametro, const string& nombreParametro)
        : edad(edadParametro), nombre(nombreParametro) {
        cout << "Constructor con parámetros llamado" << endl;
    }

    Persona(const Persona& otra) : edad(otra.edad), nombre(otra.nombre) {
        cout << "Constructor de copia llamado" << endl;
    }

    Persona(Persona&& otra) noexcept : edad(otra.edad), nombre(move(otra.nombre)) {
        cout << "Constructor de movimiento llamado" << endl;
    }

    ~Persona() {
        cout << "Destructor llamado para " << nombre << endl;
    }

    void mostrarInfo() const {
        cout << "Nombre: " << nombre << ", Edad: " << edad << endl;
    }

private:
    int edad;
    string nombre;
};
```

Clases en c++

Inicialización



```
1 class MiClase {
2 public:
3     MiClase () {};
4     MiClase (int v, bool b) : valor(v) , valor2(b) {
5     };
6
7 private:
8     int valor{};
9     bool valor2{};
10};
```

```
1     MiClase objeto;
2     MiClase objeto1(2, false);
3     MiClase objeto2{2, false};
```



```
#include <iostream>
#include <string>
```

```
class Persona {
public:
    Persona(const string& nombre, int edad)
        : nombre_(nombre), edad_(edad) {}

    void mostrarInformacion() const {
        cout << "Nombre: " << nombre_ << ", Edad: " << edad_ << endl;
    }

private:
    string nombre_;
    int edad_;
};

class Estudiante : public Persona {
public:
    Estudiante(const string& nombre, int edad, int numeroEstudiante)
        : Persona(nombre, edad), numeroEstudiante_(numeroEstudiante) {}

    void mostrarInformacionEstudiante() const {
        mostrarInformacion();
        cout << "Número de Estudiante: " << numeroEstudiante_ << endl;
    }

private:
    int numeroEstudiante_;
};
```



Herencia en c++

Herencia en c++

```
int main() {  
    Persona persona("Juan", 25);  
    persona.mostrarInformacion();  
  
    cout << "\n";  
  
    Estudiante estudiante("Maria", 20, 12345);  
    estudiante.mostrarInformacionEstudiante();  
  
    return 0;  
}
```

Polimorfismo

- En C++, los métodos virtuales son una característica clave de la programación orientada a objetos que permite la implementación de polimorfismo, lo que significa que las clases derivadas pueden proporcionar sus propias implementaciones de los métodos de la clase base.

```
class Figura {  
public:  
    virtual void dibujar() const {  
        cout << "Dibujar una figura." << endl;  
    }  
};  
  
class Circulo : public Figura {  
public:  
    void dibujar() const override {  
        cout << "Dibujar un círculo." << endl;  
    }  
};
```

```
void dibujarFigura(const Figura& figura) {  
    figura.dibujar();  
}  
  
int main() {  
    Figura figura;  
    Circulo circulo;  
  
    dibujarFigura(figura);  
    dibujarFigura(circulo);  
    return 0;  
}
```

Métodos virtuales puros - Interfaces

```
class Poligono
{
    int num_lados;
public:
    void setNumLados(int n){ num_lados = n; } /* Método concreto */
    virtual double calcularArea() = 0; /* Método virtual puro o abstracto */
    virtual double calcularPerimetro() = 0; /* Método virtual puro o abstracto */
};
```

```
class Triangulo: public Poligono
{
public:
    double calcularArea()
    {
        /* Implementacion específica de la clase Triangulo */
    }

    double calcularPerimetro()
    {
        /* Implementacion específica de la clase Triangulo */
    }
};
```

Métodos virtuales puros - Interfaces

```
class ElementoGraficoGenerico
{
    public:
    virtual void setColor(const double r, const double g, const double b) = 0;
    virtual void setPos(double xpos, double ypos) = 0;
    virtual void setLabel(const string lb) = 0;
    virtual void onClickPressed() = 0;
};
```

Herencia Múltiple

```
class ClaseA {  
    // ...  
};  
  
class ClaseB {  
    // ...  
};  
  
class ClaseDerivada : public ClaseA, public ClaseB {  
    // ...  
};
```

Sobrecarga de operadores

```
class NumeroComplejo {  
public:  
    NumeroComplejo operator+(const NumeroComplejo& otro) const {  
        // Sobrecarga del operador suma  
    }  
};
```

Miembros estáticos – Variables de clase



```
class Contador {  
public:  
    static int contadorGlobal;  
};  
  
// Acceso a miembro estático  
int Contador::contadorGlobal = 0;
```


Funciones Amigas



```
#include <iostream>

class ClaseAmiga {
private:
    int datoPrivado;

public:
    ClaseAmiga(int valor) : datoPrivado(valor) {}
    friend void funcionAmiga(const ClaseAmiga&);
};

void funcionAmiga(const ClaseAmiga& objeto) {
    cout << "Accediendo a datoPrivado desde la función amiga: "
         << objeto.datoPrivado << endl;
}

int main() {
    ClaseAmiga miObjeto(42);
    funcionAmiga(miObjeto);
    return 0;
}
```

Una función friend de una clase es una función que no pertenece a la clase, pero que tiene permiso para acceder a sus variables y funciones miembro privadas por medio de los operadores punto (.) y flecha (->)

Union en c++

```
union MiUnion {  
    int entero;  
    float flotante;  
    char caracter;  
};
```

```
datos.entero = 42;  
cout << "Entero: " << datos.entero << endl;  
  
datos.flotante = 3.14f;  
cout << "Flotante: " << datos.flotante << endl;  
  
datos.caracter = 'A';  
cout << "Caracter: " << datos.caracter << endl;
```

- En C++, un union es una construcción de lenguaje que permite almacenar diferentes tipos de datos en un mismo bloque de memoria.
- Todos los miembros de un union comparten la misma ubicación de memoria, y solo uno de ellos puede ocupar esa ubicación en un momento dado. La idea es ahorrar memoria al permitir que diferentes tipos compartan el mismo espacio.

Enums en c++

```
enum DiaSemana {  
    Lunes,  
    Martes,  
    Miercoles,  
    Jueves,  
    Viernes,  
    Sabado,  
    Domingo  
};  
  
int main() {  
    DiaSemana hoy = Miercoles;  
    if (hoy == Miercoles) {  
        cout << "Es miércoles." << endl;  
    }  
    return 0;  
}
```

- Un enum (enumeración) es una forma de definir un conjunto de constantes con nombres.
- Proporciona una manera de asignar nombres simbólicos a valores enteros, lo que hace que el código sea más legible y mantenible.

Enums en c++

```
#include <iostream>

enum class Color {
    Rojo,
    Verde,
    Azul
};

int main() {
    Color colorFavorito = Color::Verde;

    switch (colorFavorito) {
        case Color::Rojo:
            cout << "Tu color favorito es rojo." << endl;
            break;
        case Color::Verde:
            cout << "Tu color favorito es verde." << endl;
            break;
        case Color::Azul:
            cout << "Tu color favorito es azul." << endl;
            break;
    }
    return 0;
}
```

Enums y Unions

```
union MiUnion {  
    int entero;  
    float flotante;  
};  
  
struct MiStructConUnion {  
    MiUnion datos;  
    char caracter;  
};  
  
int main() {  
    MiStructConUnion* estructuraEnHeap = new MiStructConUnion;  
    estructuraEnHeap->datos.entero = 42;  
    estructuraEnHeap->caracter = 'A';  
    delete estructuraEnHeap;  
    return 0;  
}
```

- Las uniones y enums en sí mismas no tienen asignación dinámica de memoria.
- Ambas son estructuras de datos que se utilizan para organizar y representar información de manera eficiente, pero no almacenan tamaños variables como las clases o structs.
- Por lo tanto, se utilizan comúnmente en el stack.
- Aun así, pueden formar parte de estructuras más grandes que se asignan dinámicamente en el heap



Ejercicio



¿ En qué parte de la memoria se asignarán las variables ?

```
#include <iostream>
```

```
class Clase {
```

```
public:
```

```
    Clase(int val) : valor(val) {}
```

```
    void imprimir() {
```

```
        std::cout << "Valor de la clase: "
```

```
        << valor << std::endl;
```

```
    }
```

```
private:
```

```
    int valor;
```

```
};
```

```
struct Estructura {
```

```
    int valor;
```

```
};
```

```
enum Enumerado { VALOR_1, VALOR_2 };
```



Ejercicio



```
int main() {  
    Clase objetoClase(10);  
    objetoClase.imprimir();  
  
    Estructura objetoEstructura;  
    objetoEstructura.valor = 20;  
    std::cout << "Valor de la estructura: "  
                << objetoEstructura.valor << std::endl;  
  
    Enumerado enumStack = VALOR_1;  
  
    Clase* ptrClase = new Clase(30);  
    ptrClase->imprimir();  
    delete ptrClase;  
}
```

```
Estructura* ptrEstructura = new Estructura();  
ptrEstructura->valor = 40;  
std::cout << "Valor de la estructura: "  
            << ptrEstructura->valor << std::endl;  
delete ptrEstructura;  
  
Enumerado* otroEnum = new Enumerado;  
* otroEnum = VALOR_2;  
delete otroEnum;  
  
return 0;  
}
```

Programación genérica

La programación genérica en C++ es una técnica que permite escribir código que puede funcionar con diferentes tipos de datos sin tener que escribir una versión específica del código para cada tipo.

Esto se logra mediante el uso de plantillas, que son modelos de clases o funciones que pueden aceptar uno o más tipos de datos como parámetros.

Programación genérica - Funciones



```
#include <iostream>

template <typename T>
void intercambiar(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int num1 = 5, num2 = 10;
    cout << "Antes del intercambio: " << num1 << ", " << num2 << endl;
    intercambiar(num1, num2);
    cout << "Después del intercambio: " << num1 << ", " << num2 << endl;
    double pi = 3.14, e = 2.71;
    cout << "Antes del intercambio: " << pi << ", " << e << endl;
    intercambiar(pi, e);
    cout << "Después del intercambio: " << pi << ", " << e << endl;
    return 0;
}
```

Programación genérica - Clases



```
#include <iostream>

template <typename T>
class Contenedor {
private:
    T dato;

public:
    Contenedor(T valor) : dato(valor) {}

    T obtenerDato() const {
        return dato;
    }
};

int main() {
    Contenedor<int> contenedorInt(42);
    cout << "Contenedor de enteros: " << contenedorInt.obtenerDato() << endl;

    Contenedor<double> contenedorDouble(3.14);
    cout << "Contenedor de doubles: " << contenedorDouble.obtenerDato() << endl;

    Contenedor<string> contenedorString("Hola, Mundo!");
    cout << "Contenedor de cadenas: " << contenedorString.obtenerDato() << endl;

    return 0;
}
```

Programación genérica – Parámetros no-tipo



```
1 template<int N>
2 void imprimirValor(int valor) {
3     for (int i = 0; i < N; ++i) {
4         std::cout << valor << " ";
5     }
6     std::cout << std::endl;
7 }
8 imprimirValor<5>(10);
9 imprimirValor<3>(7);
```

Consideraciones de memoria

- Cuando se utilizan tipos genéricos a través de plantillas, el compilador genera código específico para cada tipo con el que se instancia la plantilla.
- El tamaño de la instancia de la plantilla depende del tamaño de los tipos utilizados.
- Se puede especializar plantillas para tipos específicos si necesitas controlar el tamaño de la instancia para ciertos tipos. Esto puede ser útil para optimizar el uso de memoria en situaciones específicas.

Consideraciones de memoria

```
template <typename T>
class Contenedor {
private:
    T dato;

public:
    // ...
};

// Especialización para tipos int
template <>
class Contenedor<int> {
private:
    int dato;

public:
    // ...
};
```



Ejercicio



Ejemplos de reserva de memoria dinámica/estática.

- Almacenar la información de datos de temperatura mínima y máxima de cada día para todo el año.
- Almacenar las calificaciones de un grupo de estudiantes donde el máximo de alumnos por comisión es fijo.

Recursividad

Algoritmos y Estructuras de datos

Prof. Ing. Gomez Pablo



29 de marzo de 2023

Temario

- Recursividad
- Recursividad Simple, Múltiple, Directa, Indirecta
- Factorial
- Fibonacci
- Fibonacci Simple
- Recursividad de Cola
- De recursivo a iterativo

Definición

Una función es aquella que se llama a si misma en su cuerpo

$$f(x) = \dots f(x) \dots$$

Condiciones de una función recursiva

- Debe existir una condición de corte de la recursión
- Se debe garantizar que la condición sea alcanzada eventualmente

Definición

Se escriben como una función definida por tramos

$$f(x) = \begin{cases} \dots & \Rightarrow \text{si se cumple condición} \\ x * fact(x - 1) & \Rightarrow \text{si no se cumple condición} \end{cases}$$

Iterative

```
def fact_iter(n):  
    total, k = 1, 1  
    while k <= n:  
        total, k = total*k, k+1  
    return total
```

$$n! = \prod_{k=1}^n k$$

Names: n, total, k, fact_iter

Recursive

```
def fact(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fact(n-1)
```

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{otherwise} \end{cases}$$

Names: n, fact

Recursión Simple

Existe una única llamada a la función en el cuerpo

Recursión Múltiple

Hay dos o más llamadas a la función en el cuerpo

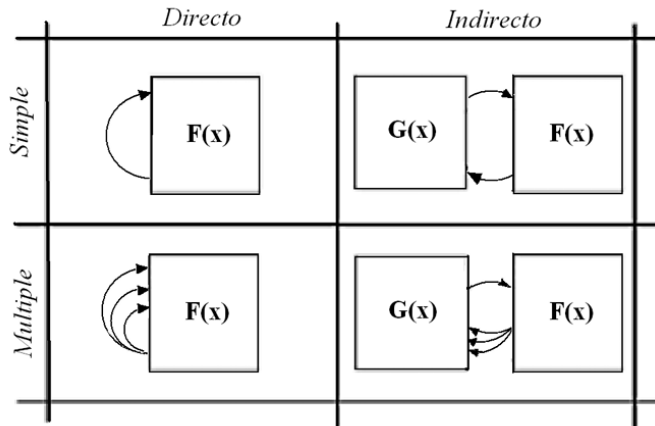
Recursión Directa

La función se llama siempre a sí misma.

Recursión Indirecta

La función llama a una segunda función que vuelve a llamar a la primera.

Tipos de recursión



Ejemplo : Función Factorial

$$fact(n) = \begin{cases} 1 & \Rightarrow n = 1 \\ n * fact(n - 1) & \Rightarrow n > 1 \end{cases}$$

Despliegue de la recursividad

$$\begin{aligned} fact(5) &= 5 * fact(4) \\ &= 5 * 4 * fact(3) \\ &= 5 * 4 * 3 * fact(2) \\ &= 5 * 4 * 3 * 2 * fact(1) \\ &= 5 * 4 * 3 * 2 * 1 \end{aligned}$$

Factorial recursivo

```
1  #include <iostream>
2  int fact( int n) {
3      if (n < 1) {error}
4      if (n==1) {
5          return 1;
6      } else {
7          return fact(n-1) * n ;
8      }
9  }
```

Ejemplo : Sucesión de Fibonacci

$$fib(x) = \begin{cases} 1 & \Rightarrow x = 1 \\ 1 & \Rightarrow x = 2 \\ fib(x-1) + fib(x-2) & \Rightarrow x > 2 \end{cases}$$

Despliegue de la recursividad

$$fib(1) = 1$$

$$fib(2) = 1 + 1 = fib(1) + fib(1) = 2$$

$$fib(3) = 1 + 2 = fib(1) + fib(2) = 3$$

$$fib(4) = 2 + 3 = fib(2) + fib(3) = 5$$

$$fib(x) = fib(x-2) + fib(x-1)$$



Fibonacci recursivo

```
1  #include <iostream>
2  using namespace std;
3  int fib(x){
4      if(x< 3){
5          return 1
6      }
7      else{
8          return fib(x-1) + fib(x-2)
9      }
10 }
11
12 int main() {
13     int result = fib(7);
14     cout<< result<<endl;
15     return 0;
16 }
```

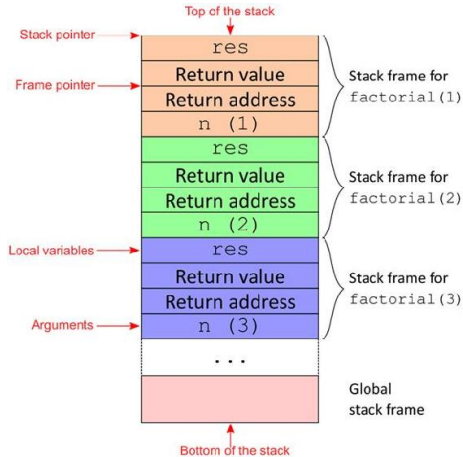
Llamado de funciones

Como ya sabemos cada vez que se llama a una función se crea una nueva entrada en la pila. Esta estructura se llama Stack Frame y consiste en los campos necesarios para continuar la ejecución luego de procesar la función.

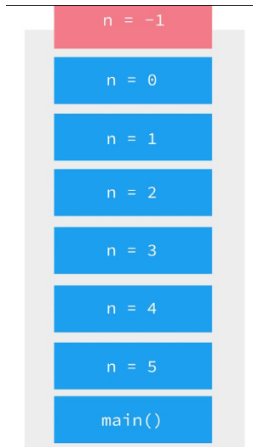
Stack Frame

- Puntero a la instrucción próxima instrucción cuando retorne.
- Todos los argumentos pasados a la función.
- Las variables locales
- El valor de retorno.

Stack frames



Stack overflow



Recursiva doble a recursiva simple

$$fib(x) = \begin{cases} 1 & \Rightarrow x < 3 \\ fib_{aux}(3, 1, 1, x) & \Rightarrow x > 3 \end{cases}$$

$$fib_{aux}(y, a_1, a_2, x) = \begin{cases} a_1 + a_2 & \Rightarrow y = x \\ fib_{aux}(y + 1, a_1 + a_2, a_1, x) & \Rightarrow y < x \end{cases}$$



Fibonacci recursivo simple

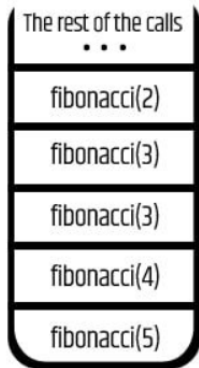
```
1  int fibaux (int y, int a1, int a2, int x){
2      if(y==x){
3          return a1 + a2;
4      } else{
5          return fibaux(y+1, a1+a2, a1, x);
6      }
7  }
8  int fib2(int x){
9      if (x<3){
10         return 1;
11     } else{
12         return fibaux(3, 1, 1, x);
13     }
14 }
```


Sólo la función auxiliar es recursiva

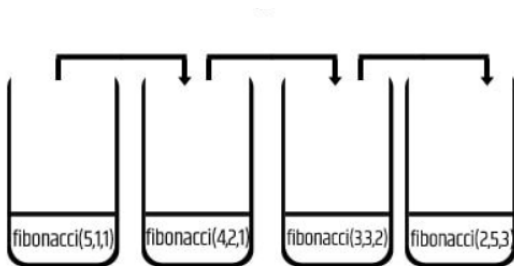
Recursión de cola

El valor que retorna la función es el llamado a la función recursiva (no sería recursiva a la cola si al resultado de la recursión, le aplicara una nueva función).

En general es conveniente dejarlo expresado como una función recursiva de cola, para poder hacer el paso a una función iterativa. En principio siempre que se tenga esta condición, se puede transformar en una función iterativa que va a ser más eficiente en términos de memoria y no hay riesgo de quedarse sin memoria del stack por excesivas llamadas a la función recursiva



Regular Recursion
Memory Usage Keeps Growing



Tail Call Optimized Recursion
Memory Usage Is Constant

De recursivo a iterativo

f es una función recursiva de cola $f : T_1 \times T_2 \Rightarrow T_3$

T_1 es producto cartesiano de los dominios que no varían de llamada en llamada recursiva

T_2 es prod cartesiano de los dominios que sí varían de llamada en llamada recursiva

T_3 es prod cartesiano de los dominios de la imagen

En la función recursiva simple tenemos las variables:

$t_1 : T_1, \quad t_2 : T_2, \quad t_3 : T_3$

De recursivo a iterativo

Se hará una transformación de f del tipo:

$$f(t_1, t_2) = \begin{cases} h(t_1, t_2) & \Rightarrow d(t_1, t_2) \\ f(t_1, s(t_1, t_2)) & \Rightarrow !d(t_1, t_2) \end{cases}$$

$d : T_1 \times T_2 \Rightarrow \text{Booleano} \Rightarrow$ devuelve true si se ha alcanzado la condición de recursión y false en caso contrario.

$h : T_1 \times T_2 \Rightarrow T_3 \Rightarrow$ caso devuelto por la función cuando se alcanza el corte (por eso devuelve variable de tipo t_3)

$s : T_1 \times T_2 \Rightarrow T_2 \Rightarrow$ es la función que describe cómo varían los dominios (t_2) de llamada en llamada (depende de t_1 y t_2 pero siempre debe devolver t_2)



De recursivo a iterativo

El programa con recursión de cola quedaría

```
1  t3 f(t1, t2){
2      if(d(t1, t2)){
3          return h(t1, t2);
4      } else{
5          return f(t1, s(t1, t2));
6      }
7  }
```

Pasando a iterativo

```
1  t3 f(t1, t2){
2      while(!d(t1, t2)){
3          t2 = s(t1, t2));
4      }
5      return h(t1, t2);
6  }
```

De recursivo a iterativo

Dada la expresión matemática de la función recursiva simple a la cola de Fibonacci:

$$fib(x) = \begin{cases} 1 & \Rightarrow x < 3 \\ fib_{aux}(3, 1, 1, x) & \Rightarrow x > 3 \end{cases}$$

$$fib_{aux}(y, a_1, a_2, x) = \begin{cases} a_1 + a_2 & \Rightarrow y = x \\ fib_{aux}(y', a'_1, a'_2, x) & \Rightarrow y < x \end{cases}$$

$T_1 : Nat$

$T_2 : Nat \times Nat \times Nat$

$T_3 : Nat$

- Identificar las variables de tipo $t_1 \Rightarrow x$
- Identificar las variables de tipo $t_2 \Rightarrow y, a_1, a_2$
- Identificar $d(t_1, t_2)$ (condición de corte de la recursión)
 $\Rightarrow y < x$
- Identificar $h(t_1, t_2)$ (valor devuelto cuando la condición de corte es alcanzada) $\Rightarrow a_1 + a_2$ (de esto se deduce que T_3 es de tipo Nat)
- Identificar $s(t_1, t_2)$ (como varían las variables de una iteración a otra) $\Rightarrow$
 $y' = y + 1$
 $a'_1 = a_1 + a_2$
 $a'_2 = a_1$
- Escribir la forma iterativa.



De recursivo a iterativo

```
1  int fib(int x){
2      if (x<3){
3          return 1;
4      } else{
5          return fib_sec(3, 1, 1, x);
6      }
7  }
8  int fib_sec(y, a1, a2, x){
9      while(y<x){
10         int aux = a1;
11         y++;
12         a1 = a1 + a2;
13         a2 = aux;
14     }
15     return a1 + a2;
16 }
```




De recursivo a iterativo

```
1  int fib(x){
2      if x<=2 {
3          return 1;
4      } else {
5          int y = 3;
6          int a1 = 1;
7          int a2 = 1;
8          while (!(y==x)) {
9              y=y+1;
10             int aux = a1;
11             a1=a1+a2;
12             a2=aux;
13         }
14         return a1+a2;
15     }
16 }
```

De recursivo a iterativo

Encontrar el mayor elemento en un arreglo

$$f(a[\text{int}]) \Rightarrow \text{int}$$

$$f(a[]) = f_{aux}(a[], i, m, size)$$

$$f_{aux}(a[], i, m, size) = \begin{cases} m & \Rightarrow i = size \\ f_{aux}(a[], i + 1, \max(m, a[i]), size) & \Rightarrow i < size \end{cases}$$

$$\max(x, y) = \begin{cases} x & \Rightarrow x > y \\ y & \Rightarrow x \leq y \end{cases}$$

$$T_1 : \text{Nat}[] \times \text{Nat}$$

$$T_2 : \text{Nat} \times \text{Nat}$$

$$T_3 : \text{Nat}$$

De recursivo a iterativo

- Identificar las variables de tipo $t_1 \Rightarrow a[], size$
- Identificar las variables de tipo $t_2 \Rightarrow i, m$
- Identificar $d(t_1, t_2) \Rightarrow i = size$
- Identificar $h(t_1, t_2) \Rightarrow m$
- Identificar $s(t_1, t_2) \Rightarrow$
 $i' = i + 1$
 $m = \max(m, a[i])$
- Escribir la forma iterativa.

```
1 int max(x, y){  
2     if(x>y) return x;  
3     else return y;  
4 }
```

De recursivo a iterativo

```
1 //Forma recursiva
2 int maxArray(a[]){
3     int size = a[].size();
4     if (size>1){
5         return maxArrayAux(a[], 0, a[0], size);
6     }
7 }
8 int maxArrayAux(a[], int i, int m, int size){
9     if(i==size){
10         return m;
11     } else{
12         return
13             maxArrayAux(a[], i+1, max(m, a[i]), size);
14     }
15 }
```



De recursivo a iterativo

```
1 //Forma iterativa
2 int maxArraySec(a[]){
3     int i = 0;
4     int m = a[0];
5     int size = a[].size();
6
7     while(i != size){
8         i++;
9         m = max(m, a[i]);
10    }
11
12    return m;
13 }
```

Colas y Pilas

Algoritmos y Estructuras de datos



3 de abril de 2025

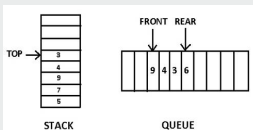
Lista

- Es una estructura de datos que almacena una cadena de datos mediante una referencia (puntero) al siguiente elemento.



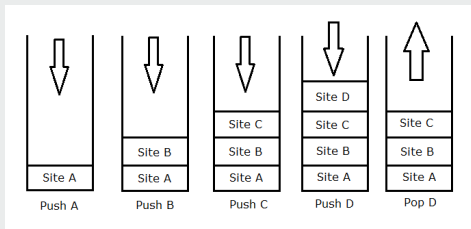
Pila (Stack) y Cola (Queue)

- Pueden ser implementadas usando arrays o listas.
- No permiten agregar o remover elementos en cualquier índice.
- Al igual que las listas son estructuras lineales contenedoras de datos. Hacemos referencia a una pila, cola de datos de tipo A.



Pila

- Es una estructura LIFO (Last in first out)
- Ej: Historial de navegación. Se navega al Sitio A $\Rightarrow$ B $\Rightarrow$ C $\Rightarrow$ D. A medida que el usuario navega se apilan los diferentes sitios en la lista de sitios. Esto asegura que el último sitio visita esté siempre en el tope de la pila. Cuando el usuario presiona el botón “Volver” se remueve el tope de la pila y se accede al sitio anterior C que quedará como tope de la pila.
- Undo/Redo



Cola

- Es una estructura FIFO (First in first out)
- Ej: Fila del supermercado. Cuando el cajero está cobrando a los clientes atenderá al primero que llegó a la fila.
- Son utilizadas como buffers. Modelo de productor y consumidor.

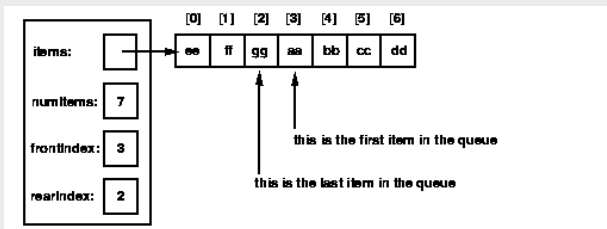
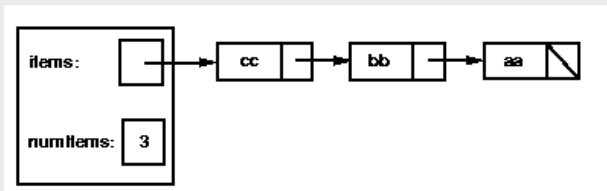


Debe respetarse el comportamiento de cada tipo FIFO/LIFO.



Pilas y Colas - Implementación

Puede utilizarse un arreglo o una lista (memoria estatica / dinamica)



Notaciones de expresiones algebraicas

Expresión algebraica

$$a - c * (d+e)$$

$$a - c * d + e \Rightarrow a - (c*d) + e$$

- Notación prefija - Polaca: operador se escribe primero y luego los operandos (+12) (*23)
- Notación infija : operador va entre medio de los dos operandos (1+2) (2*3)
- Notación posfija - Polaca inversa: operador va luego de los dos operandos (12+) (23*)



Infija vs Posfija

Infix Notation

$A + B * C \Rightarrow A + B * C \Rightarrow A + B C *$

$A + B C * \Rightarrow A + B C * \Rightarrow A B C * +$

Postfix Notation

Parenthesis

Infix Expression

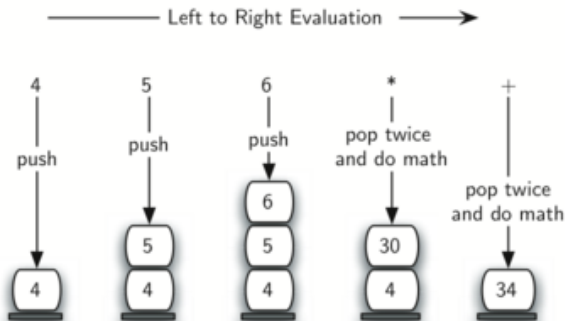
Postfix Expression

$(A + B) * C$

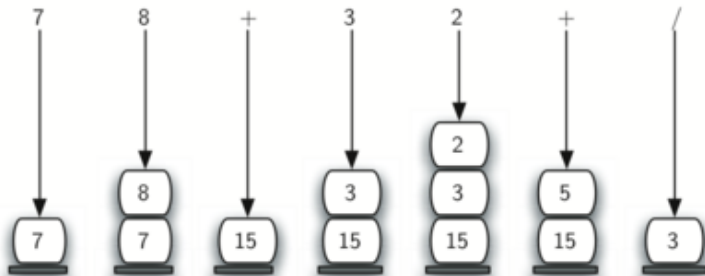
$AB + C *$

Evaluación

Tenemos la expresión posfija $4\ 5\ 6\ *\ +$

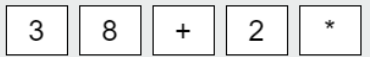


Evaluación



Calculadora Posfija Ejemplo 1

Tope



⇒ Pila con tope en 3

Tope



⇒ Desapilado [3]

Tope



⇒ Desapilado [3,8]

Tope



⇒ Desapilado [3,8,+]

Tope



⇒ Se apila resultado 11

Calculadora Posfija Ejemplo 1

Tope

2	*
---	---

 $\Rightarrow$ Desapilado [11]

Tope

*

 $\Rightarrow$ Desapilado [11,2]

Tope

 $\Rightarrow$ Desapilado [11,2,*]

Tope

22

 $\Rightarrow$ Se apila resultado 22

Tope

 $\Rightarrow$ Pila vacía

El resultado es el último valor desapilado

Calculadora Posfija - Ejemplo 2





Ejercicio

Si tenemos la expresión :

$$2 * 3 + 3 - 2 * 1 = 6 + 3 - 2 = 7 \quad (1)$$

obtenemos en posfijo:

$$2\ 3 * 3 + 2\ 1 * - \quad (2)$$

Realizar la conversión a posfijo de la siguiente expresión y su resolución:

$$2 * ((3 + 3) - 2) * 1 = 2 * 4 * 1 = 8 \quad (3)$$

Calculadora Posfija - Modelo

Pila	Pila Auxiliar
Tope 2 3 * 3 + 2 1 * -	Tope
Tope 3 * 3 + 2 1 * -	Tope 2
Tope * 3 + 2 1 * -	Tope 3 2
Tope 3 + 2 1 * -	Tope 6
Tope + 2 1 * -	Tope 3 6
Tope 2 1 * -	Tope 9
Tope 1 * -	Tope 2 9
Tope * -	Tope 1 2 9
Tope -	Tope 2 9
Tope	Tope 7

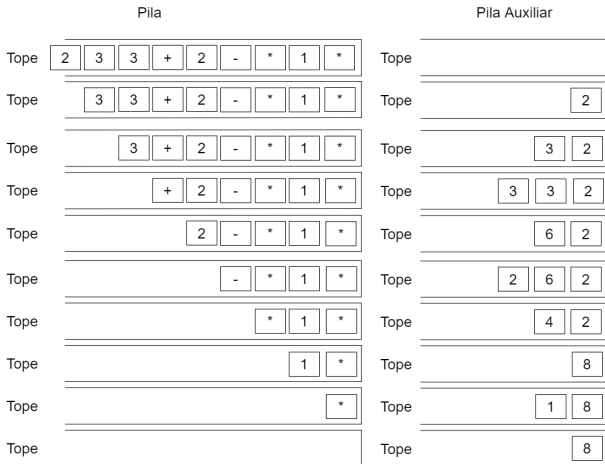
Calculadora Posfija - Resolución

$$2 * ((3 + 3) - 2) * 1$$

(1)

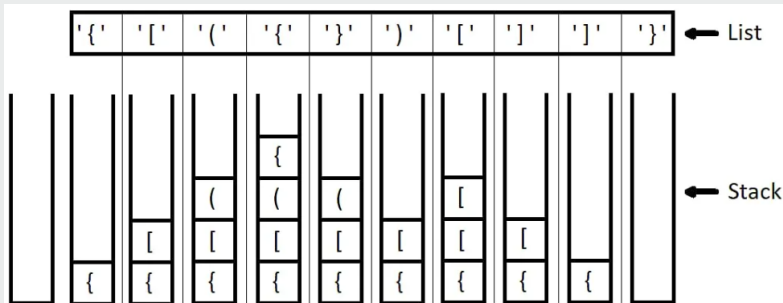
$$2 \ 3 \ 3 \ + \ 2 \ - \ * \ 1 \ *$$

(2)



Balaneo de paréntesis

- Por cada paréntesis (apilo.
- Por cada paréntesis) desapilo.
- Si la pila es vacía al final , entonces la expresión está balanceada.



Listas

Algoritmos y Estructuras de datos



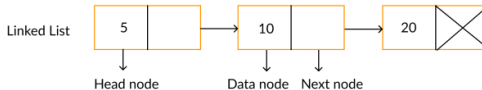
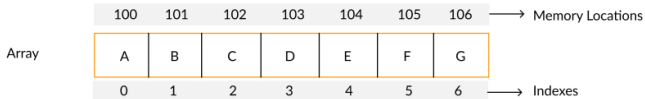
27 de marzo de 2025

Lista enlazada



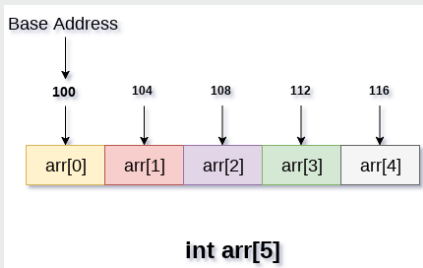
Uso de memoria

- Uso de memoria DINÁMICO
- En contrapartida con los arreglos , que tienen un numero predefinido de valores/elementos (Uso de memoria ESTÁTICO)



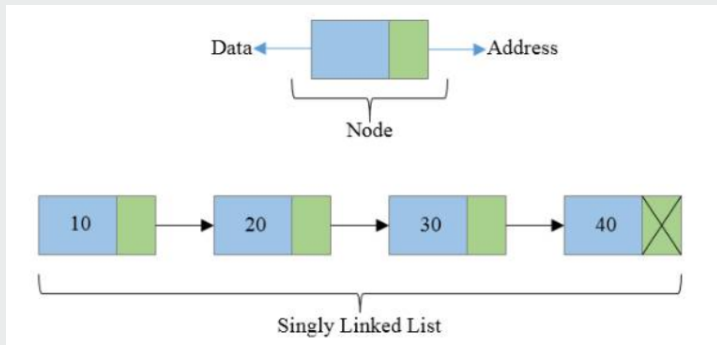
Estático

- Uso más eficiente para acceder a memoria. Acceso directo o indexado a cada dato.
- Numero fijo de elementos.
- Reservo memoria extra para garantizar cobertura hasta un máximo de datos.



Dinámico

- Acceso secuencial a memoria.
- Sin límite en la cantidad de elementos.
- Se utiliza la memoria realmente necesaria. Se solicita cada vez que se incorpora un dato.



Estructura recursiva

- Acceder al elemento $n \Rightarrow$ siguiente elemento $n-1$
- Acceder al elemento $n-1 \Rightarrow$ siguiente elemento $n-2$
- ...
- Acceder al elemento $1 \Rightarrow$ comienzo de la lista

```
1      struct Nodo{  
2          int dato;  
3          Nodo *siguiente;  
4          Nodo(int a);  
5          void imprimir();  
6      };
```

Implementación de Nodo

```
1  Nodo::Nodo(int a): dato(a), siguiente(nullptr){}
2
3  void Nodo::imprimir() {
4      std::cout << "[NODO] valor: " << dato
5                  << " siguiente : " << siguiente
6                  << std::endl;
7  }
```




Implementación de Lista

```
1  class Lista {
2  public:
3      Lista ();
4      Lista (std::Nodo *n);
5      ~Lista();
6      void agregar (int d);
7      bool es_vacia ();
8      int cabeza ();
9      Lista *resto ();
10     void imprimir();
11 private:
12     std::Nodo *comienzo;
13 };
```



Implementación de Lista

```
1  Lista::Lista() {
2      comienzo = nullptr;
3  }
4
5  Lista::Lista(std::Node *n) {
6      comienzo = n;
7  }
8
9  Lista::~~Lista() {
10     std::cout << "Llamando al destructor y libera
11     std::Node* actual = comienzo;
12     while (actual != nullptr) {
13         std::Node* siguiente = actual->siguiente;
14         delete actual;
15         actual = siguiente;
16     }
17 }
```



Implementación de Lista

```
1 void Lista::agregar(int d) {
2     auto *nuevo = new std::Nodo(d);
3     nuevo->siguiente = comienzo;
4     comienzo = nuevo;
5 }
6
7 bool Lista::es_vacia() {
8     return comienzo == nullptr;
9 }
10
11 int Lista::cabeza() {
12     if (es_vacia()) {
13         return -1;
14     }
15     return comienzo->dato;
16 }
```



Implementación de Lista

```
1  Lista *Lista::resto() {
2      if (es_vacia()) {
3          return nullptr;
4      }
5      return new Lista (comienzo->siguiente);
6  }
7
8  void Lista::imprimir() {
9      if(resto() == nullptr) {
10         return;
11     }
12     comienzo->imprimir();
13     resto()->imprimir();
14 }
```

C++ Templates - Generics

```
template <typename T>
T myMax (T x, T y)
{
    return (x > y) ? x: y;
}

int main ()
{
    count << myMax<int> (3, 7) << endl;
    count << myMax<char> ('g' , 'e') << endl;
    return 0;
}
```

*Compiler internally generates
and adds below code*

```
int myMax (int x, int y)
{
    return (x > y) ? x: y;
}
```

*Compiler internally generates
and adds below code*

```
char myMax (char x , char y)
{
    return (x > y) ? x: y;
}
```

Implementación de Nodo Generico

```
1  template <typename T>
2  struct Nodo{
3      T dato;
4      Nodo<T> *siguiente;
5      Nodo(T a);
6      void imprimir();
7  };
```

Implementación de Nodo Generico

```
1  template<typename T>
2  Nodo<T>::Nodo(T a): dato(a), siguiente(nullptr) {
3
4  template<typename T>
5  void Nodo<T>::imprimir() {
6      std::cout << "[NODO] valor: " << dato
7                  << " siguiente : " << siguiente
8                  << std::endl;
9  }
```



Implementación de Lista Generica

```
1  template <typename T>
2  class Lista {
3  public:
4      Lista ();
5      ~Lista();
6      void agregar (T d);
7      bool es_vacia ();
8      int cabeza ();
9      Lista<T> *resto ();
10     void imprimir();
11 private:
12     Lista (Nodo<T>* n);
13     Nodo<T> *comienzo;
14 };
```




Implementación de Lista Generica

```
1  template <typename T>
2  Lista<T>::Lista() {
3      comienzo = nullptr;
4  }
5
6  template <typename T>
7  Lista<T>::Lista(Nodo<T>* n) {
8      comienzo = n;
9  }
```



Implementación de Lista Generica

```
1  template <typename T>
2  Lista<T>::~~Lista() {
3      std::cout << "Llamando al destructor y libera
4      Nodo<T>* actual = comienzo;
5      while (actual != nullptr) {
6          Nodo<T>* siguiente = actual->siguiente;
7          delete actual;
8          actual = siguiente;
9      }
10 }
11
12 template <typename T>
13 void Lista<T>::agregar(T d) {
14     auto *nuevo = new Nodo(d);
15     nuevo->siguiente = comienzo;
16     comienzo = nuevo;
17 }
```



Implementación de Lista Generica

```
1  template <typename T>
2  bool Lista<T>::es_vacia() {
3      return comienzo == nullptr;
4  }
5
6  template <typename T>
7  int Lista<T>::cabeza() {
8      if (es_vacia()) {
9          return -1;
10     }
11     return comienzo->dato;
12 }
```

Implementación de Lista Generica

```
1  template <typename T>
2  Lista<T> *Lista<T>::resto() {
3      if (es_vacia()) {
4          return nullptr;
5      }
6      return new Lista<T> (comienzo->siguiente);
7  }
8
9  template <typename T>
10 void Lista<T>::imprimir() {
11     if(resto() == nullptr) {
12         return;
13     }
14     comienzo->imprimir();
15     resto()->imprimir();
16 }
```

```
1  template<typename T>
2  class Contenedor {
3  private:
4      T dato;
5  public:
6      Contenedor(): dato(0) {
7          std::cout << "number" << std::endl; };
8  };
9
10 template<>
11 class Contenedor<std::string> {
12 private:
13     std::string dato;
14 public:
15     Contenedor(): dato(
16         "NO_INICIALIZADO") { std::cout << "string" <<
```

```
1  template <typename T>
2  void imprimir(T valor) {
3      std::cout << "Template generico: "
4                  << valor << std::endl;
5  }
6
7  template <>
8  void imprimir<int>(int valor) {
9      std::cout << "Especializacion para int: "
10                << valor << std::endl;
11 }
12
13 int main() {
14     imprimir(3.14);
15     imprimir(42);
16     return 0;
17 }
```

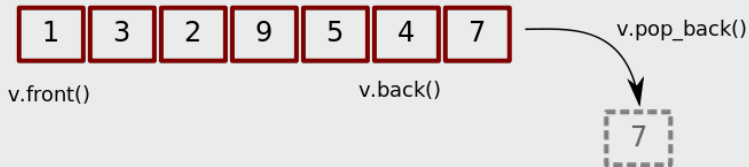
```
1  template <typename T1, typename T2>
2  class Par {
3  public:
4      void mostrar() {
5          cout << "Template generico para T1 y T2"
6              << endl;
7      }
8  };
9
10 template <typename T>
11 class Par<T, T> {
12 public:
13     void mostrar() {
14         cout << "Ambos tipos son iguales"
15             << endl;
16     }
17 }
```

Estructura similar a un arreglo pero puede cambiar de tamaño

- Se encuentran definidos en la Standard Template Library (STL)
- Tiene funciones miembro de modificación, iteración y capacidad.
- Al extender la capacidad existen dos enfoques :
Acceso mixto (seudo indexado). No se puede reservar memoria dinámicamente extendiendo el rango en la misma región de memoria. “Como una Lista de arreglos”
Almacenar por ej. 10 posiciones y en vez de sumar otras diez reservaríamos 20 posiciones y debo copiar los valores del primer vector al segundo (necesito memoria para alojar ambos - suele no ser conveniente)

Funciones miembro

- Modificación: `insert()` , `erase()`
- Iteración: `begin()` , `end()`
- Capacidad: `size()` , `empty()` , `resize()`



Ejemplo uso vector

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main()
6  {
7      vector<int> g1;
8      for (int i = 1; i <= 5; i++)
9          g1.push_back(i);
10
11     cout << "Vector: ";
12     for (vector<int>::iterator i = g1.begin();
13          i != g1.end(); ++i)
14         cout << *i << " ";
15     cout << endl ;
```

Ejemplo uso vector

```
1      cout << "Tamaño" << g1.size() << endl;
2      cout << "Capacidad" << g1.capacity() << endl;
3      cout << "Tam. Max." << g1.max_size() << endl;
4
5      g1.resize(4);
6      cout << "Tamaño: " << g1.size() << endl;
7      cout << "Vector: ";
8
9      for (vector<int>::iterator i = g1.begin();
10           i != g1.end(); ++i)
11          cout << *i << " ";
12      cout << endl;
```

```
1      if (g1.empty() == false)
2          cout << "Vector no vacio" << endl;
3      else
4          cout << "Vector vacio" << endl;
5
6      cout << "g1[2]=" << g1[2] << endl;
7      cout << "g1.at(3)=" << g1.at(3) << endl;
8      cout << "g1.front()=" << g1.front() << endl;
9      cout << "g1.back()=" << g1.back() << endl;
10
11     return 0;
12 }
```